ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

Môn học: Vi Xử Lý – Vi Điều Khiển (CO3009)
Học kỳ 1, năm học 2026-2027
TP. Hồ Chí Minh, ngày 29 tháng 10 năm 2026

THÔNG BÁO VÀ HƯỚNG DẪN THỰC HIỆN
BÀI TẬP LỚN MÔN VI XỬ LÝ – VI ĐIỀU KHIỂN

Đề tài: Hệ thống đèn giao thông 2 tuyến đường trên mạch XNUCLEO-F103RB

Tài liệu này vừa là **thông báo chính thức** về Bài tập lớn (BTL), vừa là **hướng dẫn từng bước** để các nhóm bắt đầu thực hiện. **Phần A** gồm các thông tin hành chính: nhóm, thời hạn, sản phẩm nộp, đánh giá. **Phần B** hướng dẫn kỹ thuật theo đúng trình tự nên làm, từ lúc nhận mạch đến lúc nộp bài. Các nhóm nên đọc hết Phần A, sau đó làm lần lượt từng bước ở Phần B và đánh dấu vào checklist ở cuối mỗi bước. Tài liệu này không cần nộp lại, mà chỉ là nội dung hướng dẫn cho dự án. Trong trường hợp có thắc mắc, sinh viên có thể đặt câu hỏi trong nhóm Zalo của môn học. Các file hướng dẫn, mã nguồn tham khảo cho bài tập lớn sẽ được tải lên LMS và trong cả nhóm Zalo.

Nội dung

PHẦN A. THÔNG BÁO CHUNG 1
A1. Hình thức tổ chức 1
A2. Yêu cầu của đề tài 1
A3. Thời hạn 1
A4. Sản phẩm nộp 1
A5. Tiêu chí đánh giá (dự kiến) 1
PHẦN B. HƯỚNG DẪN THỰC HIỆN 1
B0. Lộ trình tổng quát 1
Bước 1. Lập nhóm và tạo repository GitHub 1
Bước 2. Tìm hiểu phần cứng và tạo dự án STM32CubeIDE 1
Bước 3. Kiểm thử phần cứng 1
Bước 4. Hiện thực scheduler O(1) 1
Bước 5. Hiện thực các module phụ trợ 1
Bước 6. Thiết kế và hiện thực máy trạng thái 1
Bước 7. Kiểm thử tổng thể, viết báo cáo và nộp bài 1
B8. Lỗi thường gặp 1
B9. Liên hệ hỗ trợ 1

PHẦN A. THÔNG BÁO CHUNG

A1. Hình thức tổ chức

- Bài tập lớn được thực hiện theo **nhóm, tối đa 3 sinh viên/nhóm**.
- Mỗi nhóm dùng **một repository GitHub duy nhất**. Lịch sử commit là căn cứ đánh giá mức độ đóng góp của từng thành viên, vì vậy **mỗi thành viên commit bằng tài khoản của chính mình**.

A2. Yêu cầu của đề tài

Nhóm hiện thực hệ thống đèn giao thông cho ngã tư 2 tuyến đường trên mạch XNUCLEO-F103RB (chip STM32F103RB), gồm:

- **Chế độ tự động:** hai tuyến đèn Đỏ – Xanh – Vàng hoạt động luân phiên theo thời gian cài đặt.
- **Chế độ bằng tay:** người dùng chuyển trạng thái đèn bằng nút nhấn.
- **Chế độ chỉnh thời gian:** thay đổi thời gian đèn Đỏ – Xanh – Vàng bằng nút nhấn, thông tin hiển thị trên LCD.
- **Hiển thị môi trường:** đọc cảm biến DHT20, hiển thị nhiệt độ và độ ẩm trên LCD.
- **Scheduler O(1) (bắt buộc):** toàn bộ hệ thống chạy trên scheduler với SCH_Add_Task, SCH_Delete_Task, SCH_Update (O(1)) và SCH_Dispatch_Tasks; hỗ trợ one-shot task.

A3. Thời hạn

Mốc	Thời hạn
Nộp báo cáo và mã nguồn	*04/12/2026*
Demo	*Tuần 15/16 – theo lịch thông báo*

A4. Sản phẩm nộp

STT	Sản phẩm	Yêu cầu
1	Báo cáo (PDF)	Viết theo Mau_BaoCao_BTL_CO3009.docx; trang bìa có thông tin thành viên (họ tên, MSSV, nhóm) và link GitHub. Tên file: CO3009_MSSV1_MSSV2_..._Report.pdf
2	Mã nguồn	Link GitHub, ghi rõ commit/tag nộp bài. Repo public hoặc thêm giảng viên làm collaborator. Có README.md hướng dẫn build và nạp
3	Demo	Demo trực tiếp trên mạch XNUCLEO-F103RB; lịch demo thông báo sau

A5. Tiêu chí đánh giá (dự kiến)

STT	Tiêu chí	Trọng số
1	Scheduler: đầy đủ các hàm, SCH_Update đạt O(1), hỗ trợ one-shot task	[10]%
2	Thiết kế máy trạng thái, chuyển đổi giữa các chế độ hợp lý và an toàn	[30]%
3	Chức năng đèn giao thông: tự động, bằng tay, chỉnh thời gian	[20]%
4	Module phụ trợ: software timer, button, DHT20, LCD	[10]%
5	Tổ chức mã nguồn: tách lớp phần cứng/phần mềm, quy ước đặt tên, lịch sử commit	[10]%
6	Báo cáo và demo	[20]%

Bài nộp trễ hạn [quy định trừ điểm/không chấp nhận]. Các trường hợp sao chép mã nguồn hoặc báo cáo giữa các nhóm sẽ bị xử lý theo quy định của Nhà trường.

PHẦN B. HƯỚNG DẪN THỰC HIỆN

B0. Lộ trình tổng quát

Bài tập lớn nên được làm theo từng lớp, từ dưới lên: **phần cứng chạy đúng → scheduler chạy đúng → các module phụ trợ chạy đúng → máy trạng thái**. Mỗi lớp cần được kiểm thử độc lập trước khi xây lớp tiếp theo. Lỗi phát hiện ở lớp dưới sớm sẽ tiết kiệm rất nhiều thời gian gỡ lỗi về sau.

Tuần	Công việc	Kết quả cần có
Tuần 1	Bước 1 – 3: lập nhóm, tạo repo, tạo dự án, kiểm thử phần cứng	LED nháy, 4 trạng thái đèn, 4 nút, I2C scan thấy 2 thiết bị
Tuần 2	Bước 4: hiện thực và kiểm thử scheduler	Nhiều task chạy đúng chu kỳ, one-shot tự xóa
Tuần 3	Bước 5: software timer, button, đèn, DHT20, LCD	Mỗi module có demo nhỏ chạy trên scheduler
Tuần 4	Bước 6: thiết kế và hiện thực 3 máy trạng thái	Hệ thống hoàn chỉnh
Tuần 5	Bước 7: kiểm thử tổng thể, viết báo cáo, chuẩn bị demo	Báo cáo PDF, tag nộp bài

[Hướng dẫn] *Lộ trình mang tính gợi ý; các nhóm điều chỉnh theo mốc thời gian ở mục A3.*

Bước 1. Lập nhóm và tạo repository GitHub

Nhóm trưởng tạo repository, thêm các thành viên làm collaborator, sau đó mỗi thành viên clone về máy:

```
git clone https://github.com/<tai-khoan>/<ten-repo>.git
cd <ten-repo>
git checkout -b dev            # nhánh phát triển chung
git checkout -b feature/scheduler   # mỗi phần việc một nhánh riêng
```

Thêm file **.gitignore** ở thư mục gốc để không đưa các file sinh ra khi build lên GitHub:

```
Debug/
Release/
*.o
*.d
*.elf
*.map
*.su
.metadata/
```

- Thống nhất quy ước đặt tên ngay từ đầu (ví dụ: tên file chữ thường, nối bằng gạch dưới: fsm_automatic.c; tên hàm theo dạng <module>_<hành_động>: button_read(), traffic_set_state()).
- Commit thường xuyên, mỗi commit một thay đổi có ý nghĩa, thông điệp rõ ràng (ví dụ: "scheduler: add one-shot task support").

- Tham khảo cấu trúc dự án mẫu:
  https://github.com/nhanksd85/STM32F103_MCU/blob/scheduler

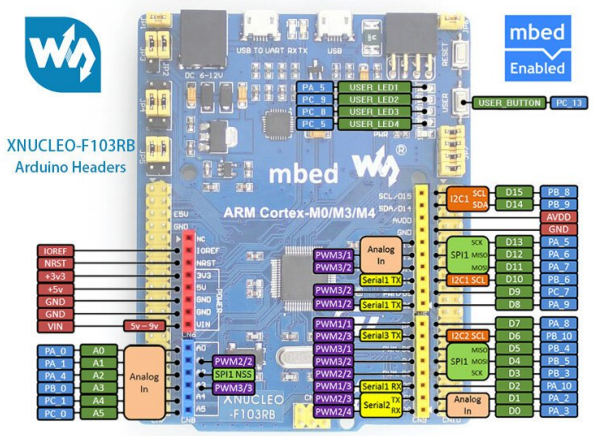
Checklist hoàn thành bước này

- ☐ Repo đã tạo, đủ thành viên, có .gitignore và README.md
- ☐ Đã đăng ký nhóm và link GitHub trên LMS
- ☐ Nhóm đã thống nhất quy ước đặt tên và phân công công việc

Bước 2. Tìm hiểu phần cứng và tạo dự án STM32CubeIDE

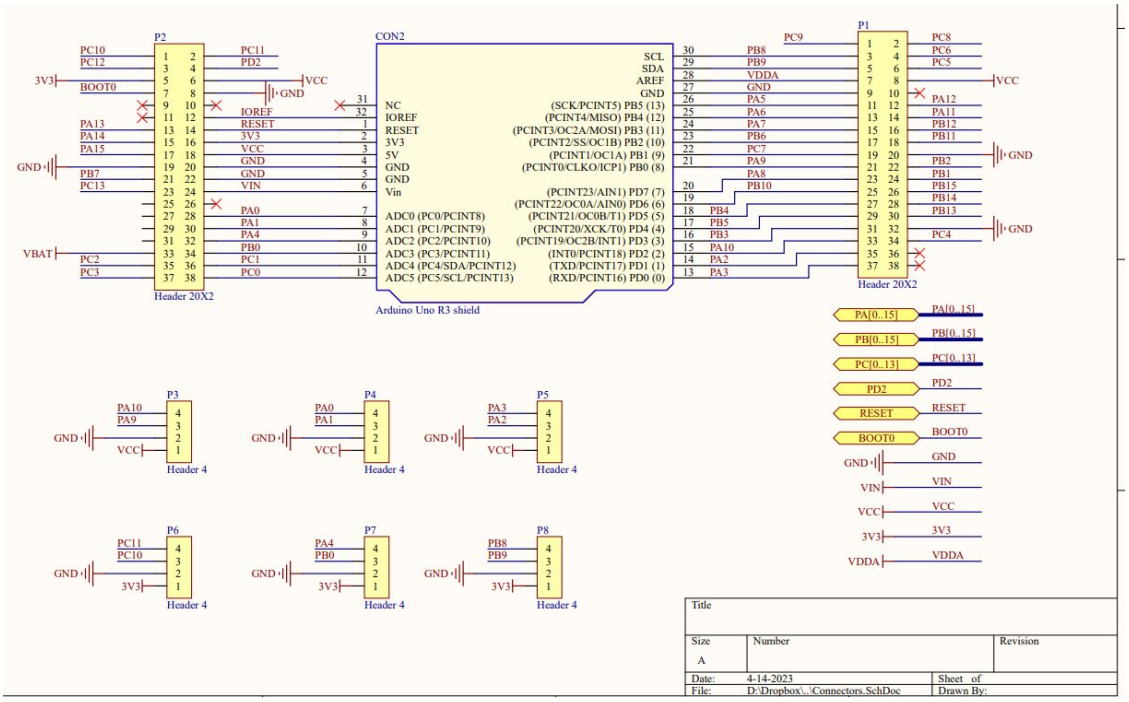
2.1 Mạch XNUCLEO-F103RB

Dự án phát triển trên mạch XNUCLEO-F103RB (chip STM32F103RB). Đây là thông tin quan trọng khi tạo dự án trên STM32CubeIDE.



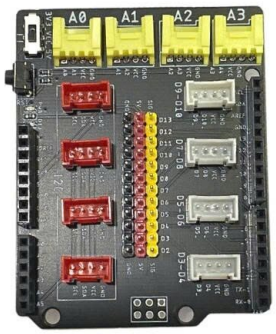
Hình 1: Mạch XNUCLEO-F103RB

Các chân trên khe cắm tương thích Arduino có tên **Dx** (x là thứ tự chân). Mỗi chân Dx nối với một chân của STM32, ví dụ D2 nối với PA10. Sinh viên **tự tra schematic** dưới đây để tìm chân STM32 tương ứng.



Hình 2: Schematic hàng chân Arduino

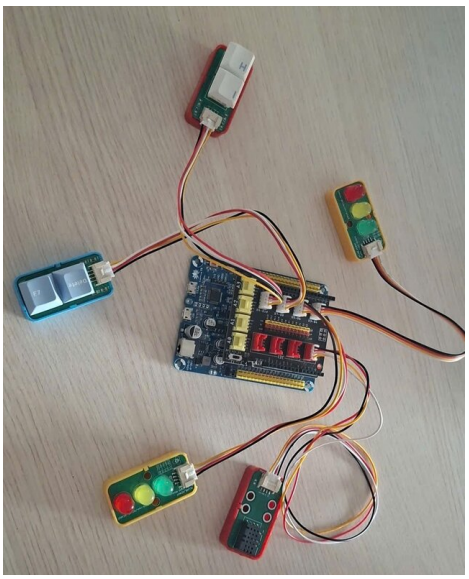
Mạch XNUCLEO được gắn thêm một mạch mở rộng chân theo chuẩn GROVE. Mỗi khe GROVE có 2 chân tín hiệu.



Hình 3: Mạch mở rộng chân theo chuẩn GROVE

2.2 Kết nối thiết bị

Thiết bị	Vị trí kết nối	Ghi chú
Module đèn giao thông 1	D3 – D4	2 tín hiệu → 4 trạng thái
Module đèn giao thông 2	D5 – D6	2 tín hiệu → 4 trạng thái
Module nút nhấn 1	D7 – D8	2 nút, nối trực tiếp GPIO
Module nút nhấn 2	D9 – D10	2 nút, nối trực tiếp GPIO
Cảm biến DHT20, LCD	Khe I2C (màu đỏ)	Cắm khe nào cũng được
LED có sẵn / nút USER	PA5, PC9 / PC13	Có sẵn trên mạch



Hình 4: Hệ thống sau khi kết nối hoàn chỉnh

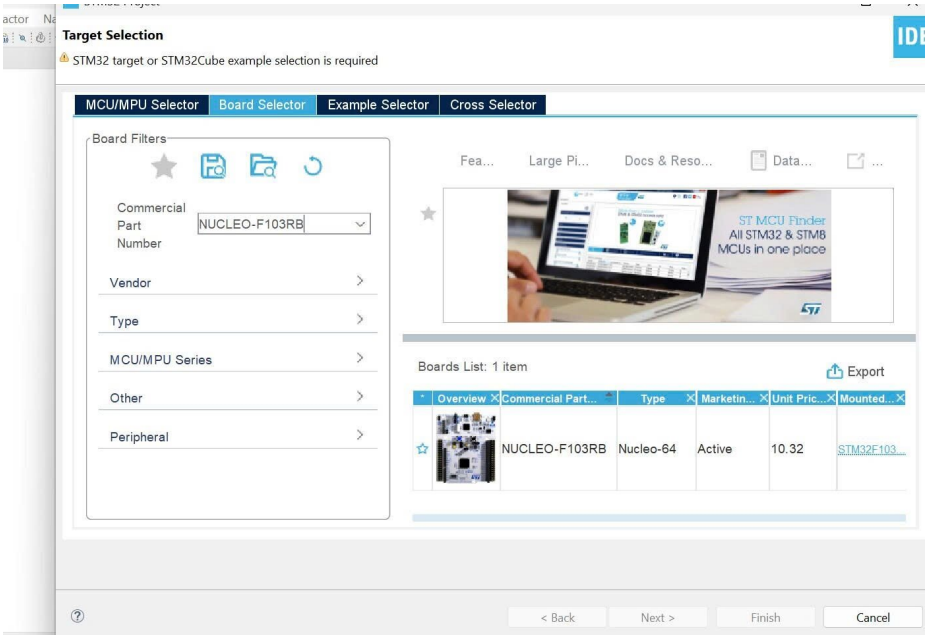
Dựa vào schematic, nhóm điền bảng tra chân sau đây. Bảng này dùng lại trong mục 3.1 của báo cáo.

Chân Arduino	Chân STM32	Chế độ GPIO	User Label đặt trong .ioc
D3	*[Pxx]*	Output Push-Pull	*[...]*
D4	*[Pxx]*	Output Push-Pull	*[...]*
D5	*[Pxx]*	Output Push-Pull	*[...]*
D6	*[Pxx]*	Output Push-Pull	*[...]*
D7	*[Pxx]*	Input	*[...]*
D8	*[Pxx]*	Input	*[...]*
D9	*[Pxx]*	Input	*[...]*
D10	*[Pxx]*	Input	*[...]*

2.3 Tạo dự án trên STM32CubeIDE

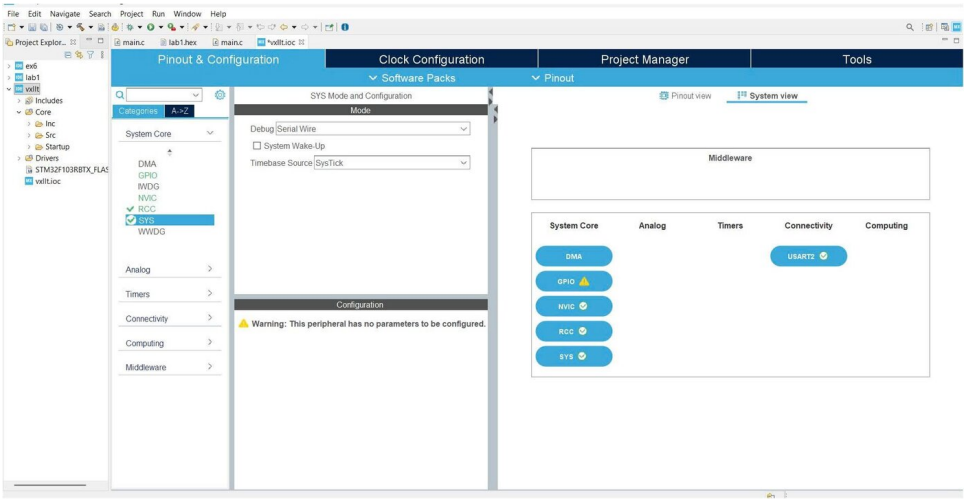
> **LƯU Ý QUAN TRỌNG**
> XNUCLEO-F103RB dùng chip **STM32F103RB**, khác với chip STM32F103C6 trong môi trường mô phỏng. Nếu cấu hình sai, mạch **chỉ nạp được một lần** và sau đó không nạp được nữa. Thực hiện chính xác hai bước dưới đây.

Bước 2a: Tạo dự án mới, chuyển sang thẻ **Board Selector**, tìm và chọn board **NUCLEO-F103RB**.



Hình 5: Chọn board NUCLEO-F103RB trong Board Selector

Bước 2b: Mở file .ioc, vào **Pinout & Configuration → System Core → SYS**. Mục **Debug** bắt buộc chọn **Serial Wire**.



Hình 6: Cấu hình Debug là Serial Wire

Nếu mạch đã bị lỗi nạp, nhóm thực hiện lại đủ hai bước trên, build lại chương trình và làm theo video: https://www.youtube.com/watch?v=qxaW6FMiVWQ. Nếu vẫn không nạp được, liên hệ giảng viên để được hỗ trợ.

2.4 Cấu hình ban đầu trong file .ioc

- Cấu hình các chân GPIO theo bảng tra chân ở mục 2.2 và đặt User Label dễ hiểu (ví dụ: TL1_A, TL1_B, BTN_1…).
- Bật **I2C1** cho DHT20 và LCD (kiểm tra trên schematic khe I2C màu đỏ nối với cặp chân SCL/SDA nào).
- Cấu hình một timer (ví dụ TIM2) tạo ngắt chu kỳ **10 ms** làm nhịp (tick) cho scheduler, và bật ngắt timer trong NVIC.

Chu kỳ ngắt được tính theo công thức T = (PSC + 1) × (ARR + 1) / f_TIM. Ví dụ với f_TIM = 8 MHz, chọn PSC = 7999 và ARR = 9 sẽ cho T = 10 ms. Nhóm cần kiểm tra **f_TIM thực tế** trong thẻ Clock Configuration của dự án và tính lại PSC, ARR cho phù hợp.

Checklist hoàn thành bước này

- ☐ Dự án tạo từ Board Selector với NUCLEO-F103RB, Debug = Serial Wire
- ☐ Bảng tra chân D3 – D10 đã điền đầy đủ
- ☐ GPIO, I2C1, timer 10 ms đã cấu hình; dự án build không lỗi
- ☐ Đã commit dự án gốc lên GitHub

Bước 3. Kiểm thử phần cứng

Trước khi viết scheduler hay máy trạng thái, nhóm cần chắc chắn từng thiết bị hoạt động đúng. Các đoạn mã kiểm thử ở bước này chỉ dùng tạm, viết trực tiếp trong vòng while(1) của main.c.

3.1 Kiểm tra nạp chương trình: nháy LED PA5

```
while (1) {
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    HAL_Delay(500);
}
```

3.2 Xác định tổ hợp tín hiệu cho module đèn

Mỗi module đèn được điều khiển bởi 2 chân, cho 4 trạng thái: Xanh, Vàng, Đỏ và Tắt. Nhóm lần lượt xuất 4 tổ hợp, quan sát và điền bảng:

```
// Thử lần lượt 4 tổ hợp (0,0) (0,1) (1,0) (1,1), mỗi tổ hợp giữ 2 giây
for (int i = 0; i < 4; i++) {
    HAL_GPIO_WritePin(TL1_A_GPIO_Port, TL1_A_Pin, (i >> 1) & 1);
    HAL_GPIO_WritePin(TL1_B_GPIO_Port, TL1_B_Pin, i & 1);
    HAL_Delay(2000);
}
```

Tín hiệu A	Tín hiệu B	Trạng thái đèn quan sát được
0	0	*[...]*
0	1	*[...]*
1	0	*[...]*
1	1	*[...]*

3.3 Kiểm tra nút nhấn

Đọc từng nút và hiển thị kết quả qua LED PA5 để xác định mức logic khi nhấn (0 hay 1). Thông tin này cần cho module button ở Bước 5.

```
if (HAL_GPIO_ReadPin(BTN_1_GPIO_Port, BTN_1_Pin) == GPIO_PIN_RESET)
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
else
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
```

3.4 Quét thiết bị I2C

Quét toàn bộ địa chỉ I2C để xác nhận DHT20 và LCD đã kết nối, đồng thời tìm địa chỉ của LCD (địa chỉ LCD khác nhau tùy module). Đặt breakpoint hoặc dùng LED để xem kết quả.

```
uint8_t found[8]; int n = 0;
for (uint8_t addr = 1; addr < 128; addr++) {
    if (HAL_I2C_IsDeviceReady(&hi2c1, addr << 1, 1, 10) == HAL_OK)
        if (n < 8) found[n++] = addr;   // đặt breakpoint để xem mảng found
}
```

Checklist hoàn thành bước này

- ☐ LED PA5 nháy, nạp lại chương trình nhiều lần không lỗi
- ☐ Bảng tổ hợp tín hiệu đèn đã xác định cho cả 2 module
- ☐ Biết mức logic khi nhấn của cả 4 nút và nút USER
- ☐ Quét I2C thấy 2 thiết bị; đã ghi lại địa chỉ DHT20 và LCD

Bước 4. Hiện thực scheduler O(1)

Scheduler là phần **bắt buộc** và là nền tảng cho toàn bộ hệ thống. Từ bước này, **không dùng HAL_Delay** trong chương trình chính; mọi công việc được chạy dưới dạng task.

4.1 Cấu trúc dữ liệu và các hàm

```
typedef struct {
    void   (*pTask)(void);   // con trỏ tới hàm task
    uint32_t Delay;           // số tick còn lại (tương đối)
    uint32_t Period;          // chu kỳ, 0 = one-shot task
    uint8_t  RunMe;           // cờ báo task sẵn sàng chạy
    uint32_t TaskID;
} sTask;

void     SCH_Init(void);
uint32_t SCH_Add_Task(void (*pFunc)(void), uint32_t DELAY, uint32_t PERIOD);
uint8_t  SCH_Delete_Task(uint32_t taskID);
void     SCH_Update(void);          // gọi trong ngắt timer
void     SCH_Dispatch_Tasks(void);  // gọi trong while(1)
```

Hàm SCH_Update bắt buộc đạt độ phức tạp **O(1)**: chỉ xử lý task đầu danh sách.

```
void SCH_Update(void) {
    if (SCH_count == 0) return;
    if (SCH_tasks_G[0].Delay > 0) SCH_tasks_G[0].Delay--;
    if (SCH_tasks_G[0].Delay == 0) SCH_tasks_G[0].RunMe = 1;
}
```

> **Ý TƯỞNG CHÍNH**
>
> Để SCH_Update chỉ cần xét task đầu tiên, danh sách task phải được **sắp xếp theo thời điểm chạy**, và mỗi task lưu **Delay tương đối** so với task đứng trước nó. Khi đó, công việc nặng được dồn sang SCH_Add_Task: tìm đúng vị trí chèn, trừ Delay của task mới cho tổng Delay các task phía trước, và cập nhật lại Delay của task ngay phía sau.
>
> Trong SCH_Dispatch_Tasks: khi task đầu có RunMe = 1, gọi hàm task, xóa task khỏi đầu danh

> sách; nếu Period > 0 thì thêm lại với Delay = Period, nếu Period = 0 (one-shot) thì không thêm lại.
>
> Nhóm tự chọn cài đặt bằng Array hay Linked List và giải thích lựa chọn trong báo cáo.

4.2 Kiểm thử scheduler

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) SCH_Update();
}

// trong main(), trước while(1):
SCH_Init();
SCH_Add_Task(toggle_PA5, 0, 50);     // 500 ms (tick 10 ms)
SCH_Add_Task(toggle_PC9, 0, 100);    // 1000 ms
SCH_Add_Task(one_shot_test, 300, 0); // chạy 1 lần sau 3 s
while (1) { SCH_Dispatch_Tasks(); }
```

Checklist hoàn thành bước này

- ☐ Hai LED nháy đúng chu kỳ 0,5 s và 1 s khi chạy đồng thời
- ☐ One-shot task chạy đúng 1 lần và đã bị xóa khỏi danh sách
- ☐ SCH_Delete_Task xóa được task bất kỳ mà các task còn lại vẫn đúng thời điểm
- ☐ Đã thử thêm task với nhiều giá trị Delay khác nhau (chèn đầu, giữa, cuối danh sách)

Bước 5. Hiện thực các module phụ trợ

Các module ở bước này thường **phụ thuộc phần cứng** và là nơi duy nhất được phép gọi hàm HAL. Mỗi module gồm một cặp file .h/.c, cung cấp các hàm cho lớp máy trạng thái sử dụng.

Module	File gợi ý	Nhiệm vụ và lưu ý
Software timer	software_timer.h/.c	Các bộ đếm thời gian dùng cho máy trạng thái (setTimer, cờ timeout); cập nhật bởi một task chu kỳ 10 ms
Button	button.h/.c	Đọc nút mỗi 10 ms, chống dội (so sánh nhiều lần đọc liên tiếp), phát hiện nhấn và nhấn giữ; cung cấp hàm kiểu isButtonPressed(i)
Đèn giao thông	traffic_light.h/.c	Hàm đặt trạng thái cho từng tuyến, ví dụ traffic_set(1, RED), dùng bảng tổ hợp ở mục 3.2
DHT20	dht20.h/.c	Giao tiếp I2C, đọc và chuyển đổi nhiệt độ, độ ẩm theo datasheet; đọc với chu kỳ hợp lý (không đọc liên tục)
LCD	lcd.h/.c	Khởi tạo, xóa màn hình, đặt con trỏ, in chuỗi/số; dùng địa chỉ tìm được ở mục 3.4

Mỗi module nên có một chương trình thử nhỏ chạy trên scheduler. Ví dụ: nhấn nút 1 thì đổi trạng thái LED; mỗi 2 giây đọc DHT20 và in lên LCD.

Checklist hoàn thành bước này

- ☐ Nhấn nút không bị nhận nhiều lần do dội phím

☐ Điều khiển được từng tuyến đèn qua hàm của module đèn
☐ LCD hiển thị nhiệt độ, độ ẩm cập nhật định kỳ
☐ Không còn lệnh HAL nào nằm ngoài các module phụ trợ và main.c

Bước 6. Thiết kế và hiện thực máy trạng thái

6.1 Thiết kế trước, lập trình sau

Nhóm vẽ máy trạng thái trên giấy hoặc công cụ vẽ (draw.io, Graphviz…) và thống nhất trong nhóm trước khi viết mã. Gợi ý chia thành 3 máy trạng thái lớn:

- **fsm_automatic:** các trạng thái kết hợp của 2 tuyến (ví dụ: Đỏ1–Xanh2, Đỏ1–Vàng2, Xanh1–Đỏ2, Vàng1–Đỏ2), chuyển trạng thái khi software timer hết hạn.
- **fsm_manual:** người dùng chuyển trạng thái đèn bằng nút nhấn.
- **fsm_tuning:** chọn đèn cần chỉnh, tăng/giảm thời gian, xác nhận; LCD hiển thị giá trị đang chỉnh.

LCD có thể được dùng như một menu để hiển thị và lựa chọn chế độ.

> **PHẦN TRỌNG TÂM KHI ĐÁNH GIÁ**
> Với mỗi lần chuyển giữa các chế độ lớn (ví dụ từ tự động sang bằng tay), nhóm cần trả lời được: đèn 2 tuyến ở trạng thái nào ngay sau khi chuyển; các timer và task được khởi tạo lại ra sao; LCD hiển thị gì; và làm thế nào để **không bao giờ có 2 tuyến cùng đèn Xanh**.
> Khi chỉnh thời gian, cần quy định ràng buộc hợp lệ giữa thời gian Đỏ, Xanh, Vàng và cách xử lý khi người dùng nhập giá trị không hợp lệ.

6.2 Quy tắc hiện thực

- Mỗi máy trạng thái là một cặp file .h/.c riêng; hàm chạy máy trạng thái được gọi như một task của scheduler.
- Máy trạng thái **KHÔNG chứa lệnh truy xuất phần cứng** (HAL_GPIO_…, HAL_I2C_…). Mọi thao tác đèn, nút, LCD đều thông qua hàm của các module ở Bước 5.
- Các biến dùng chung (trạng thái hiện tại, thời gian Đỏ/Xanh/Vàng) khai báo trong global.h/global.c.

```
/* fsm_automatic.c – khung gợi ý */
#include "fsm_automatic.h"

void fsm_automatic_run(void) {
    switch (status) {
    case AUTO_RED1_GREEN2:
        traffic_set(1, RED); traffic_set(2, GREEN);
        if (timer_flag[0] == 1) {
            status = AUTO_RED1_AMBER2;
            setTimer(0, time_amber);
        }
        if (isButtonPressed(0))  { status = MAN_INIT; }
        break;
    // TODO: các trạng thái còn lại
    default: break;
    }
}
```

Cấu trúc thư mục gợi ý:

```
Core/Inc/   global.h  scheduler.h  software_timer.h  button.h
            traffic_light.h  dht20.h  lcd.h
            fsm_automatic.h  fsm_manual.h  fsm_tuning.h
Core/Src/   các file .c tương ứng
```

Checklist hoàn thành bước này

- ☐ Sơ đồ 3 máy trạng thái và máy trạng thái chuyển chế độ đã vẽ xong
- ☐ Hệ thống chạy đủ 3 chế độ, chuyển chế độ đúng thiết kế
- ☐ Không có lệnh HAL trong các file fsm_*.c
- ☐ Đã thử các tình huống biên: nhấn nút liên tục, chuyển chế độ đúng lúc đèn đang Vàng, nhập thời gian không hợp lệ

Bước 7. Kiểm thử tổng thể, viết báo cáo và nộp bài

1. Chạy hệ thống liên tục ít nhất [..] phút để kiểm tra độ ổn định.
2. Viết báo cáo theo Template_Report_BTL_CO3009.docx; xóa các đoạn hướng dẫn màu xám trước khi xuất PDF.
3. Cập nhật README.md: mô tả dự án, cấu trúc thư mục, cách build và nạp, cách vận hành.
4. Gộp mã nguồn vào nhánh main, gắn tag nộp bài và ghi commit hash vào trang bìa báo cáo:

```
git checkout main
git merge dev
git tag v1.0-final
git push origin main --tags
```

5. Nộp file CO3009_Nhom<XX>_Report.pdf và link GitHub trên LMS trước hạn.

B8. Lỗi thường gặp

Hiện tượng	Cách xử lý
Nạp được 1 lần, sau đó không nạp được nữa	Kiểm tra Board Selector (NUCLEO-F103RB) và SYS → Debug = Serial Wire; làm theo video khắc phục ở mục 2.3
Task không chạy hoặc chạy sai chu kỳ	Kiểm tra đã gọi HAL_TIM_Base_Start_IT, đã bật ngắt timer trong NVIC, PSC/ARR tính đúng với f_TIM
Chương trình bị treo sau một thời gian	Kiểm tra tràn mảng task, task được thêm liên tục mà không xóa, HAL_Delay còn sót trong task
Nhấn nút một lần nhưng nhận nhiều lần	Xem lại cơ chế chống dội và việc xóa cờ nút sau khi xử lý
LCD không hiển thị	Chạy lại I2C scan để xác nhận địa chỉ; nhớ dịch trái 1 bit địa chỉ 7-bit khi gọi hàm HAL
Đèn hiển thị sai màu	Đối chiếu lại bảng tổ hợp tín hiệu ở mục 3.2 và thứ tự chân trong .ioc

B9. Liên hệ hỗ trợ

Mọi thắc mắc về nội dung, sinh viên đặt câu hỏi trên diễn đàn môn học để các nhóm khác cùng tham khảo. Trường hợp mạch không thể nạp lại chương trình hoặc hư hỏng phần cứng, sinh viên liên hệ trực tiếp giảng viên.

Tài liệu đính kèm: Project_Group.docx (đặc tả chi tiết), Template_Report_BTL_CO3009.docx (mẫu báo cáo).